

Plan de Dezvoltare și Cerințe (C/C++ pe Linux)

1. Obiective, Cerințe Funcționale și Non-Funcționale (Revizuite)

Obiectivele principale rămân aceleași, dar implementarea se va concentra pe eficiența oferită de **I/O non-blocant** și **multithreading**.

Cerințe Funcționale de Bază (CF)

ID	Cerință Funcțională	Implementare C/C++ (Linux)	Prioritate
CF-100	Interceptare Trafic (Proxy Core)	Utilizarea apelurilor de sistem de rețea (socket, bind, listen, accept). Utilizarea epoll pentru I/O non-blocant de înaltă performanță.	Înaltă
CF-101	Suport HTTPS (SSL/TLS)	Utilizarea librăriei OpenSSL pentru stabilirea conexiunilor TLS și decriptarea traficului (man-in-the-middle).	Medie
CF-200	Vizualizare/Logare	Logarea detaliilor traficului într-un fișier sau trimiterea lor către un modul de UI/monitorizare folosind IPC .	Înaltă
CF-300	Editare Headere/Conținut	Logica de parsare și manipulare a pachetelor HTTP (header-e și body) implementată manual sau cu o librărie C/C++	Înaltă

		lightweight (ex: httparser).	
CF-400	Motor de Reguli	Implementarea logicii de căutare și înlocuire (CF-401, CF-402) folosind librăria PCRE (Perl Compatible Regular Expressions) pentru performanță.	Înaltă

Cerințe Non-Funcționale (CNF) - Accent pe Sistem

ID	Cerință Non-Funcțională	Detalii de Implementare în C/C++
CNF-100	Performanță	Utilizarea epoll pentru a gestiona mii de conexiuni simultan. Minimizarea copiilor de date între buffer-e.
CNF-200	Multithreading	Utilizarea Pthreads (pthread) pentru a paralela sarcinile (ex: un thread pentru epoll, pool-uri de thread-uri pentru procesarea datelor).
CNF-300	Comunicare Inter-Proces (IPC)	Utilizarea Socket-uri Unix Domain sau Cozi de Mesaje POSIX pentru a trimite date de trafic de la Proxy Core (procesul cu privilegii) la un proces de monitorizare/UI.
CNF-400	Robustete	Gestionarea erorilor de apeluri de sistem și a semnalelor (signal, sigaction). Utilizarea RAII (Resource Acquisition Is Initialization) în C++ pentru gestionarea resurselor (soclu, memorie).

2. Arhitectura Modulului și Tehnologii

Aplicația va fi structurată în cel puțin **două procese** principale pentru a separa funcționalitățile și pentru o mai bună securitate.

A. Arhitectura (Procese și Thread-uri)

1. Proxy Core (Proces 1 - Privilegiat/High Performance):

- **Scop:** Ascultă portul, acceptă conexiuni, face I/O și aplică regulile de modificare.
- **Tehnologie:** C (pentru control maxim) sau C++ cu focus pe structuri de date eficiente.
- **Model Threading:**
 - **Thread Principal:** Rulează bucla **epoll** (gestionarea I/O).
 - **Pool de Thread-uri:** Pentru sarcini CPU-intensive (parțarea HTTP, aplicarea regulilor Regex complexe).

2. Monitor/Logare (Proces 2 - UI/Management):

- **Scop:** Primește datele brute de trafic de la Core și le loghează/afisază. Gestionează regulile utilizatorului.
- **Tehnologie:** C++ (pentru o interfață de reguli mai ușor de implementat)
- **IPC:** Comunică cu Proxy Core prin mecanisme IPC.

B. Tehnologii și Librării Cheie

Componentă	Tehnologie/Librărie	Utilizare (Apeluri de Sistem)
I/O Non-Blocant	epoll (Linux kernel API)	Baza performanței. Înlocuiește select/poll.
SSL/TLS	OpenSSL	Implementarea funcțiilor de server SSL și client SSL.
Multithreading	Pthreads (ISO C standard library)	Crearea și gestionarea thread-urilor, utilizarea mutex-urilor, semafoarelor și variabilelor condiționale pentru sincronizare.
IPC	Socket-uri Unix Domain sau Cozi de Mesaje POSIX	Comunicarea structurată a datelor de trafic între Proxy Core și procesul de Logare/UI.
Regex	PCRE2	Motorul rapid pentru implementarea filtrelor de conținut (CF-401, CF-402).

3. Plan de Dezvoltare (Fazele pe Linux/C/C++)

Planul se axează pe construirea funcționalității de bază la nivelul cel mai de jos (rețea și I/O).

Fază	Focus Principal	Sarcini Cheie (C/C++ & Apeluri Sistem)
Faza 1: Fundația	Proxy I/O & Sockets	1. Implementarea Proxy Core (Proces 1) - socket, bind, listen. 2. Configurarea epoll pentru a asculta socket-ul. 3. I/O Loop de bază: citirea cererii (request) și trimiterea ei către destinație (fără parsare).
Faza 2: Multithreading & IPC	Paralelism & Comunicare	1. Integrarea Pthreads (pool de thread-uri) pentru a prelucra datele primite de epoll. 2. Implementarea IPC (ex: Socket-uri Unix) pentru a trimite cererile brute către un proces de logare.
Faza 3: Protocol & Date	Parsare HTTP & SSL	1. Logica de parsare de bază a headerelor HTTP (funcții C pentru extragerea cheie-valoare). 2. Integrarea OpenSSL (CF-101) pentru a gestiona handshake-urile TLS și decriptarea/criptarea.
Faza 4: Motor de Modificare	Reguli & PCRE	1. Implementarea structurii de date pentru stocarea regulilor în memorie. 2. Integrarea PCRE2 pentru a căuta și înlocui conținutul în buffer-ul de răspuns (body). 3. Logica de modificare a link-urilor (CF-402) aplicată după decodarea content-ului (ex: Gzip).
Faza 5: Finalizare & Hardening	Stabilitate & Optimizare	1. Testare sub sarcină (ab, wrk) pentru a optimiza performanța I/O și a thread-urilor. 2. Gestiunea resurselor (închiderea corectă a socket-urilor, close, free, delete).

